Robust Atomic Multicast: Evaluating ACK-Gated Skeen via Deterministic Protocol Validation

Agatha Schneider
Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)
Porto Alegre, Brazil
htaschneider@icloud.com

Abstract*—Atomic multicast is a central communication abstraction for partitioned state machine replication and distributed key-value stores because it orders operations only at the partitions they access while preserving consistency across overlapping destination sets. Skeen's decentralized atomic multicast protocol provides a compact timestamp-based ordering mechanism, but later work shows that the original protocol admits unsafe executions when overlapping multicasts expose real-time dependencies before all destinations have converged on the same delivery barrier. This paper presents an implementation and evaluation of two protocols in a distributed key-value store: the original Skeen protocol and the ACK-gated extension proposed by Pacheco et al. The system supports generic N-partition routing, configurable membership, arbitrary multicast destination sets, in-memory and HTTP transports, Docker deployment, and structured logging. Our main engineering contribution is a deterministic validation framework that controls message delivery using scheduled transport queues, predicate-based release rules, delivery trace recording, and cycle detection. The framework reproduces a representative unsafe execution for original Skeen without sleeps or timing races and shows that the ACK-gated variant prevents premature delivery for that same scheduled counterexample. This is counterexample reproduction and regression validation, not exhaustive safety verification. We also benchmark both protocols for cluster sizes 2, 3, and 5 and destination sets up to the full cluster under injected delays of 0, 1, 5, and 10 ms. The results quantify the cost of the acknowledgement barrier: the strengthened protocol adds measurable CPU and allocation overhead in the zero-delay case, while the extra control-message dependency becomes visible under artificial phase-counting latency.

I. Introduction

Distributed key-value stores and partitioned state machine replication systems must often coordinate operations that access only a subset of partitions. A single-key update may involve one partition, whereas a range query or multi-key transaction may span several partitions. Broadcasting every operation to every server is simple but wasteful; routing each operation only to its destination set avoids unnecessary processing at unrelated partitions but requires a multicast layer that preserves a consistent order whenever destination sets overlap.

Atomic multicast addresses this requirement by ensuring that processes delivering the same messages deliver them in the same order. Skeen's protocol is a classic decentralized solution based on logical timestamps, local timestamp proposals, a final timestamp decision, and delivery queues ordered by final sequence numbers [1]. Its appeal is that it avoids a single sequencer and lets each multicast destination participate in timestamp selection.

However, Pacheco et al. show that global total order alone is not sufficient for partitioned state machine replication when real-time dependencies cross overlapping groups [2]. In particular, the original protocol can deliver a message at one destination, accept a later overlapping multicast, and then allow another destination to observe the later message before the earlier one. The resulting execution may be compatible with each local delivery queue but incompatible with the stronger atomic global order needed to avoid application-level coordination.

This paper evaluates that issue through an independent implementation rather than only a proof sketch. We implement both original Skeen and the ACK-gated strengthened variant in a distributed key-value store, reproduce a known unsafe execution using a deterministic scheduler, and measure the cost of adding the acknowledgement barrier. The validation result is intentionally scoped: it demonstrates that the counterexample can occur in executable code and that the implemented ACK gate blocks the same schedule, but it does not replace the formal safety argument in prior work or exhaustively explore all possible interleavings.

The contributions are:

- A configurable distributed key-value store with generic partition routing, static membership configuration, arbitrary multicast destination sets, and selectable original or strengthened Skeen modes.
- A dual transport stack with an in-memory transport for tests and controlled simulation, an HTTP transport for deployment, Docker configuration, and logging abstractions.
- A deterministic validation framework that reproduces a representative unsafe original-Skeen execution using scheduled message delivery, predicate-based release, delivery recording, and cycle detection.
- A benchmark pipeline that collects Go benchmark output, parses it into CSV matrices, and generates Matplotlib plots for per-operation latency, memory consumption, allocation rate, and latency sensitivity.

The evaluation is organized around three research questions. **RQ1**: Can the unsafe execution described for original Skeen be reproduced as a deterministic counterexample in an independent implementation? **RQ2**: Does the ACK-gated extension

block that same scheduled counterexample under an identical deterministic message schedule? **RQ3**: What CPU, memory, allocation, and latency overheads does the acknowledgement barrier introduce in the measured configurations?

II. Background and Related Work

Atomic multicast provides reliable, consistently ordered delivery to a subset of processes. If two multicast messages are delivered at two common destinations, those destinations must deliver them in the same relative order. This property is especially useful in partitioned storage systems: each operation is multicast only to the partitions whose state it may read or write, but overlapping operations still observe a common serialization order.

Skeen's protocol assigns total order without a centralized sequencer [1]. A client or coordinator sends a START message to every destination in the multicast set. Each destination increments a logical clock, stores the request in a delivery queue as undeliverable, and returns a proposed timestamp. After proposals have been collected, the maximum proposed timestamp becomes the final timestamp. The final timestamp is multicast to all destinations, and a process may deliver the message when it is stable at the head of its queue. Ties are broken by process identifiers, producing a deterministic ordering relation over timestamp pairs.

The vulnerability identified by Pacheco et al. is not simply a disagreement about the final timestamp of one message [2]. The problem appears when real-time dependencies interact with overlapping destination sets. Suppose message m is delivered at one destination before a later message m' is multicast, but another destination learns enough timestamps to deliver m' before m. The local delivery rules can remain internally consistent while the global execution graph contains a cycle: a real-time edge $m \rightarrow m'$ and a delivery-order edge $m' \rightarrow m$. This violates the stronger ordering needed by partitioned state machine replication to execute delivered operations immediately without additional synchronization.

The ACK-gated extension strengthens delivery by adding an explicit acknowledgement barrier after the final timestamp is learned. Once a destination learns the final timestamp, it broadcasts an ACK to the other destinations. A message is deliverable only after the process has received acknowledgements from every destination in the multicast set. This barrier prevents a destination from delivering a finalized message while another destination in the same multicast is still able to assign or expose a lower ordering constraint for an overlapping execution.

III. System Architecture and Implementation

A. Distributed Key-Value Store

The implementation exposes a partitioned key-value store over the multicast abstraction. Requests carry an operation type, request identifier, and explicit destination set. PUT operations target exactly one partition, while range operations may target arbitrary destination sets. The routing layer supports configurable N-partition clusters, so the same protocol code can be evaluated for $N = 2$, $N = 3$, and $N = 5$ without hard-coded participant assumptions.

Each partition owns a Skeen protocol instance, a local key-value store, a logical clock, and a map of request states. A request state records the request payload, the local timestamp proposal, all collected proposals, the final timestamp, ACK state, delivery status, result data, and a completion channel used by synchronous client submission. Protocol mode is selected at construction time: original mode ignores ACK state during delivery, while strengthened mode requires every destination ACK before delivery.

B. Protocol Execution

The implementation uses four protocol message types: START, LOCAL_TS, FINAL_TS, and ACK. On START, a destination validates membership in the destination set, advances its clock, stores a local proposal, and broadcasts that proposal. When proposals from all destinations are present, the maximum timestamp is learned as the final timestamp and broadcast. A final timestamp also advances local clock state if necessary.

Delivery is attempted after every protocol message. The node collects finalized, undelivered requests and sorts them by final timestamp. A candidate can be delivered only if it is finalized, not already delivered, and lower than every other pending local or final timestamp known at the node. In strengthened mode, the same candidate must also have ACKs from all destinations. Delivery executes the key-value operation, records any range result, invokes the delivery hook used by tests, and releases the waiting submitter.

C. Dual Transport Stack and Deployment

The transport interface contains a single protocol send operation. The in-memory transport maps partition identifiers directly to local Skeen instances and is used for unit tests, benchmarks, and deterministic simulation. The HTTP transport serializes protocol messages as JSON and sends them to each peer partition's internal protocol endpoint. This separation lets the protocol be tested without network nondeterminism while preserving a deployable HTTP path. The repository also includes Docker Compose configurations for multi-process execution and logging helpers that can be disabled during benchmarks.

Both transports invoke the same protocol state machine. In the in-memory case, a send to partition identifier p performs a map lookup and calls the receiver's protocol handler directly. In the HTTP case, the same START, LOCAL_TS, FINAL_TS, and ACK records are JSON encoded and delivered to the peer endpoint configured for partition p, where the receiver invokes the same handler. The transport changes serialization and process placement, but not timestamp selection, request-state transitions, ACK bookkeeping, or delivery predicates. This equivalence is why deterministic and zero-delay experiments use the in-memory transport while deployment can use the HTTP transport.

D. Deterministic Validation Framework

Distributed protocol tests frequently rely on sleeps, goroutine scheduling, or best-effort network delays. Those techniques make unsafe traces difficult to reproduce and can confuse protocol bugs with timing accidents. Our validation framework instead introduces a scheduled transport that records outbound protocol messages rather than delivering them immediately. Tests explicitly release queued messages through predicates over message type, request identifier, sender, receiver, and destination set.

Let a queued message have fields to, $type$, id, $from$, and dst, where dst is the request destination set. A release predicate is a Boolean function over these fields:

$$\begin{aligned} P(q) ::= {} & P_{\text{type}}(type) \wedge P_{\text{id}}(id) \\ & \wedge P_{\text{from}}(from) \wedge P_{\text{to}}(to) \\ & \wedge P_{\text{dst}}(dst). \end{aligned}$$

Any conjunct may be omitted. The scheduler operation

$$\text{releaseFirst}(P)$$

removes the first queued message satisfying P and invokes the receiver's protocol handler; drain repeatedly releases the queue head until no messages remain. For example, the predicate used to release a proposal from partition 0 to partition 1 for request m is

$$q.type = \text{LOCAL\_TS} \wedge q.id = m \wedge q.from = 0 \wedge q.to = 1.$$

This definition makes the counterexample schedule independent of wall-clock time and operating-system goroutine order.

The framework also records per-partition delivery traces. These traces are converted into a directed graph: if a partition delivers a before b, the graph records $a \to b$. Tests may add real-time edges from the scenario under study, such as $m \to m'$ when m' is submitted only after m has already been delivered. A depth-first cycle detector then determines whether the combined delivery and real-time constraints violate atomic global order.

The reproduced unsafe schedule uses three partitions. Message m targets partitions 0 and 1; message m' targets partitions 0 and 2. By controlling initial clocks and releasing selected LOCAL_TS messages, the original protocol allows partition 1 to deliver m, then allows partition 0 to deliver m' before m. Adding the real-time edge from the delivery of m to the later multicast of m' creates the expected cycle. Running the same schedule in strengthened mode leaves partition 1 unable to deliver m until the missing destination ACK is released; after the scheduler drains all messages, no cycle is detected. This experiment is a precise regression trace, not a model checker: it demonstrates the existence of the original bug and the absence of that bug on this trace, while broader schedule-space exploration remains future work.

IV. Experimental Evaluation

A. Methodology

The benchmark pipeline begins with Go benchmarks using `testing.B` and `ReportAllocs`. Raw benchmark text is converted to CSV by a custom parser that extracts benchmark name, cluster size, protocol mode, destination count, fixed delay, ACK-specific delay, `ns/op`, `B/op`, and `allocs/op`. The plotting stage loads these matrices and generates high-resolution PNG plots with Matplotlib. Each figure is generated directly from parsed CSV output of the corresponding Go benchmark family, with no manual data entry. Go's benchmark harness chooses the inner iteration count for each run; our reported statistics aggregate repeated benchmark runs after normalizing by Go's per-operation counters.

Two benchmark families are used. The first evaluates destination-set overhead for $N \in \{2, 3, 5\}$ and destination counts from one partition to the full cluster; Table I reports 10 repeated runs. The second fixes $N = 3$ and injects artificial latency of 0, 1, 5, and 10 ms into protocol messages; Table II reports 5 repeated runs. A separate ACK-delay benchmark applies delay only to ACK messages, isolating the cost of the strengthened protocol's additional barrier. The current N-sweep stops at 5 because the benchmark campaign was designed to validate the small-cluster correctness scenarios and quantify local protocol overhead, not to claim asymptotic scalability. The growth from $N = 3$ to $N = 5$ motivates future measurements at $N = 7$, $N = 10$, and beyond.

B. Correctness Results

The deterministic validation tests answer the scoped forms of RQ1 and RQ2. Original Skeen reproduces the unsafe Pacheco-style trace: one partition delivers m, a later overlapping multicast m' is introduced, and another partition delivers m' before m. The resulting graph contains a cycle once the real-time edge $m \to m'$ is included. A control test also verifies that original Skeen does not create a pure delivery-order cycle without the real-time edge, matching the nature of the reported vulnerability.

The ACK-gated implementation prevents the same trace. Under the identical message schedule, a destination that has learned a final timestamp cannot deliver until every destination in the multicast set has acknowledged that final timestamp. This blocks the premature delivery that creates the original cycle. Once the scheduler drains all messages, delivery proceeds consistently and the cycle detector reports no violation. This result should be read as deterministic counterexample reproduction and regression testing rather than exhaustive verification of the strengthened protocol.

C. Zero-Delay Overhead

Table I summarizes the in-memory destination-overhead benchmark. With full destination sets, strengthened mode increases latency from 0.293 ms to 0.350 ms at $N = 2$, from 0.488 ms to 0.608 ms at $N = 3$, and from 0.751 ms to 1.193 ms at $N = 5$. The cost is also visible in memory and allocation counters because ACK state and ACK messages add per-request bookkeeping. The reported standard deviations are small for the zero-delay benchmark, so the full-destination latency differences are well above run-to-run noise in this dataset.

TABLE I
IN-MEMORY FULL-DESTINATION OVERHEAD. VALUES ARE MEAN ± SAMPLE STANDARD DEVIATION OVER 10 REPEATED BENCHMARK RUNS.

Mode / N	dst	ms/op	B/op	allocs/op
Original / 2	2	0.293 ± 0.001	5039.2 ± 2.8	53.1 ± 0.3
Strengthened / 2	2	0.350 ± 0.001	7214.4 ± 2.6	71.0 ± 0.0
Original / 3	3	0.488 ± 0.002	8074.1 ± 1.6	72.0 ± 0.0
Strengthened / 3	3	0.608 ± 0.002	12292.4 ± 3.2	104.1 ± 0.3
Original / 5	5	0.751 ± 0.003	16207.6 ± 4.3	110.0 ± 0.0
Strengthened / 5	5	1.193 ± 0.003	26320.0 ± 4.5	176.0 ± 0.0

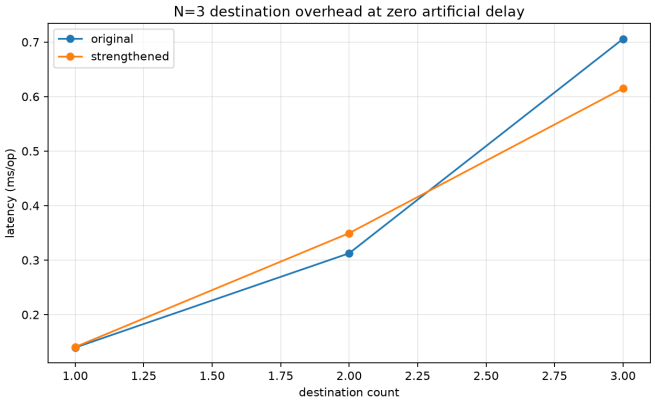

Fig. 1. Destination-set overhead for $N = 3$ without artificial latency.

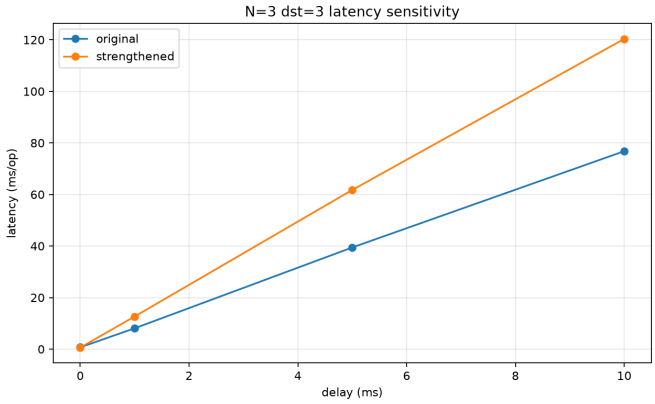

Fig. 2. $N = 3$, dst=3 sensitivity to artificial protocol-message delay.

Figure 1 shows that the ACK gate has almost no effect for single-destination operations, where the process can acknowledge itself, but the gap grows with destination-set size. This matches the implementation structure: the strengthened protocol only adds meaningful network fan-out when more than one destination participates.

D. Latency Sensitivity

Table II and Fig. 2 evaluate $N = 3$, destination count 3, and fixed artificial delay on all protocol messages. The injected delay is a phase-counting tool: it applies a constant delay to each message and intentionally excludes jitter, tail latency, packet loss, and congestion effects. Therefore the results should not be interpreted as a deployed-network performance model. At 5 ms injected delay, original Skeen averages 39.436 ms/op while strengthened Skeen averages 61.753 ms/op. At 10 ms, the original protocol averages 76.789 ms/op and the strengthened protocol averages 120.357 ms/op. The additional delay is consistent with the extra ACK dependency: the strengthened protocol is not only performing more local bookkeeping, it must also wait for another control-message phase before delivery can release the client operation.

The zero-delay row in the latency benchmark is noisier than the dedicated in-memory benchmark because it is collected through the latency benchmark harness; therefore, Table I is the better baseline for CPU-only protocol overhead. The delayed rows are nevertheless useful because the same harness applies the same delay policy to both modes.

E. ACK-Specific Delay

Figure 3 isolates ACK latency for the strengthened protocol. With no regular-message delay and only ACK messages delayed, the average operation time rises from 0.610 ± 0.005 ms at 0 ms ACK delay to 5.673 ± 0.064 ms at 1 ms, 29.648 ± 1.045 ms at 5 ms, and 58.860 ± 2.389 ms at 10 ms. In this implementation with destination count 3, the strengthened request may issue six peer ACK sends, since each of the three destinations sends ACKs to the other two destinations. A least-squares fit over the four ACK-delay means gives

$$\text{latency} \approx 0.256 + 5.860 \cdot \text{ackDelay}$$

with $R^2 = 0.9998$, close to the expected six-delay slope. This turns the plot into a mechanism check: the measured latency increase is explained by the ACK sends placed on the critical path of the in-memory benchmark harness.

V. DISCUSSION, LIMITATIONS, AND CONCLUSION

The results show a clear safety-performance trade-off. Original Skeen is smaller and faster because delivery depends on timestamp stability alone. The strengthened protocol adds explicit coordination at the multicast layer, which prevents the reproduced unsafe execution and lets the key-value layer execute delivered operations without adding its own cross-partition barrier. In low-latency in-memory settings, this cost is measurable but moderate. Under injected phase-counting latency, the additional ACK phase becomes the dominant difference.

TABLE II

$N = 3$, DST=3 LATENCY SENSITIVITY UNDER FIXED ARTIFICIAL MESSAGE DELAY. VALUES ARE MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER 5 REPEATED BENCHMARK RUNS.

Mode	delay (ms)	ms/op	B/op	allocs/op
Original	0	0.706 ± 0.492	8076.2 ± 13.8	71.8 ± 0.4
Strengthened	0	0.615 ± 0.014	12301.6 ± 27.5	104.2 ± 0.4
Original	1	8.085 ± 0.003	12581.6 ± 1.5	126.0 ± 0.0
Strengthened	1	12.670 ± 0.009	18357.2 ± 4.0	176.0 ± 0.0
Original	5	39.436 ± 0.058	12733.0 ± 11.7	127.0 ± 0.0
Strengthened	5	61.753 ± 0.113	18495.8 ± 11.0	177.8 ± 0.4
Original	10	76.789 ± 0.066	12773.2 ± 81.4	128.0 ± 0.0
Strengthened	10	120.357 ± 0.321	18594.2 ± 24.0	178.8 ± 0.4

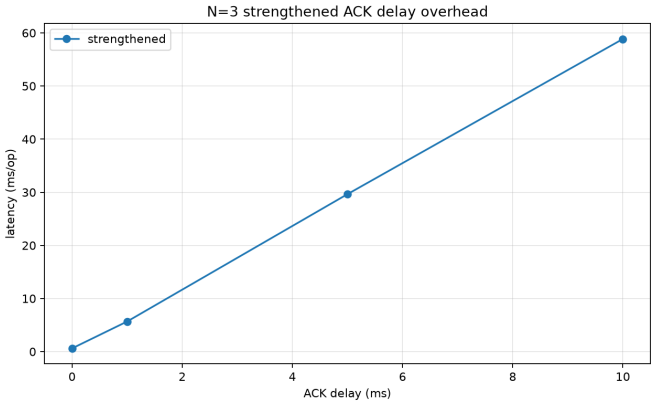

Fig. 3. $N = 3$, dst=3 overhead when artificial delay is applied only to ACK messages.

This design is attractive when the application requires the stronger ordering property described by Pacheco et al. [2]. Without the ACK gate, the storage layer would need to compensate by coordinating during execution, which moves complexity out of the multicast layer and into every replicated operation path. The strengthened protocol centralizes that responsibility in one delivery predicate.

The evaluation has several limitations. The implementation models one process per partition and assumes static membership. It does not implement replicated partition groups, crash recovery, persistent logs, view changes, or dynamic reconfiguration. The deterministic scheduler validates carefully selected traces rather than exhaustively exploring all schedules. The performance study is limited to $N \leq 5$ and artificial delay injection; larger clusters, real network jitter, packet loss, and failure recovery would need separate experiments.

A. ACK-Gate Liveness

The ACK gate strengthens safety by delaying delivery until every destination has acknowledged the final timestamp, but this also places ACK receipt on the liveness path. In the current fail-stop-free implementation, an ACK that is lost, delayed indefinitely, or never sent because a destination crashes can stall delivery forever for that request. There is no timeout-based suspicion, view change, retransmission protocol, persistent recovery, or quorum substitute for the all-destination ACK predicate. This limitation is a direct consequence of evaluating the protocol in a static, reliable-membership setting. A production design would need to combine the ACK gate with failure detection, membership reconfiguration, and recovery rules that preserve the strengthened ordering property while preventing permanent starvation.

In conclusion, the project demonstrates that deterministic scheduling is a practical way to reproduce subtle distributed coordination bugs. By controlling message release with predicates and checking delivery graphs for cycles, the framework reproduces an unsafe original-Skeen execution without relying on sleeps or probabilistic races. The ACK-gated variant prevents that scheduled execution by requiring all destinations to acknowledge final timestamps before delivery. Benchmarks show that this safety improvement carries measurable overhead in local execution and substantial latency sensitivity when ACK messages are delayed, making the trade-off explicit for designers of partitioned state machine replication systems.

REFERENCES

[1] D. Skeen, "A decentralized formalism for agreement protocols," in *Proc. IEEE Symp. Reliability in Distributed Software and Database Systems*, 1981, pp. 123–134.

[2] N. Pacheco *et al.*, "Correcting and enhancing decentralized atomic multicast algorithms," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1420–1432, 2023.